

8. 指令系统

2026 春

杨莉萍
信息学院大数据系

本章主要内容

- 指令格式
- 指令和数据的寻址方式
- 典型指令
- RISC 与 CISC
- MIPS指令系统。

指令系统基本概念

- (机器)指令
 - 计算机能直接识别、执行的某种操作命令.
- 指令系统 (“指令集”)
 - 是一台计算机中所有机器指令的集合.
 - 是机器硬件设计的依据, 也是软件设计的基础
 - 是硬件和软件间的界面, 直接影响计算机系统性能。

5.2 指令格式 (重)

- 表示一条指令的机器字, 称为**指令字**, 简称为**指令**:



- **指令格式**: 用二进制代码表示指令的结构形式。解决——
 - 指令要求计算机对数据做什么处理? ---由指令的**操作码**解决
 - 指令要求计算机处理什么数据? ---由指令的**操作数 (地址码)**解决
 - 计算机怎样找到要处理的数据? ---由指令的**寻址方式** 解决。

指令助记符

- 由于硬件只能识别1和0，所以采用二进制操作码是必要的，但是我们用二进制来书写程序却非常麻烦。
- 为了便于书写和阅读程序, 每条指令的操作码通常用3个或4个英文缩写字母来表示。这种缩写码叫做**指令助记符**：
 - 不同的计算机系统 助记符的规定不一样。常见的：ADD、SUB、MOV、JMP、STR、LDA。

指令分类方法

- 按计算机系统的层次结构分类(P153图):
 - 微指令、机器指令、汇编语言指令、高级语言指令.
- 按操作数物理位置分类：
 - 存储器 - 存储器 (SS)型: 源、目的操作数 均在主存中
 - 寄存器 - 寄存器 (RR)型: 均在CPU的寄存器中
 - 寄存器 - 存储器 (RS)型: 源操作数 在寄存器中, 目的操作数 在主存中.
- 按操作数个数分类:
 - 三地址指令、二地址、单地址、零地址。

指令分类方法

■ 根据指令的功能分类

- ◆ 传送指令 MOV、PUSH/POP、IN/OUT等
- ◆ 定点算术运算指令 ADD、SUB、INC、CMP、MUL等
- ◆ 位运算指令 NOT、AND、OR、SHL、SAL等
- ◆ 控制转移指令 JMP、JNE、CALL、RET等

(详见***MIPS*** 指令系统)。

5.2.1 指令长度

- 指令(字)长度: 一条指令包含二进制代码的位数。常称为**指令字长**。
- **机器字长**: 计算机能同时直接处理的二进制数据的位数。
(注意区分两者)
 - 有: 单字长指令、半字长指令 (指令字长等于半个机器字长)、多字长指令
- **多字长指令** 的优缺点:
 - 优: 提供足够的地址位来解决访问内存任何单元的寻址问题;
 - 缺: 必须两次或多次访问内存以取出一整条指令, 降低了CPU的执行速度, 又占用了更多的存储空间。

5.2.2 地址码

- 根据一条指令中有几个操作数地址，可将该指令称为 **几操作数指令** 或 **几地址指令**。
 - 三地址指令
 - 二(双)地址指令
 - 单地址指令
 - 零地址指令。

5.2.2 地址码

■ 三地址指令

□ 指令格式如下：



操作码op; 第1源操作数A1;第2源操作数A2; 结果A3

功能描述：

- $(A1) \text{ op } (A2) \rightarrow A3$
- $(PC) + 1 \rightarrow PC$ 。注：**PC**为程序计数器, 其中保存当前指令的地址。

□ 其格式如下：



操作码op,第一(且: 目的)操作数 **A1**,第二操作数A2

□ 功能描述：

□ $(A1) \text{ op } (A2) \rightarrow \underline{(A1)}$

□ $(PC)+1 \rightarrow PC$

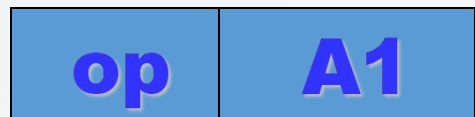
□ 二地址指令使用广泛, 但在用时必须注意: 指令执行之后, A1中原存的数据 已被新的运算结果替代了。

□ 二地址指令 根据操作数的物理位置分为：

慢 ↑
快

- ◆ SS 存储器-存储器类型
- ◆ RS 寄存器-存储器类型
- ◆ RR 寄存器-寄存器类型。

□ 指令格式为：



操作码op, 第一(且:目的)操作数A1

功能描述：

- ① 单目运算: $op(A1) \rightarrow A1$
- 单目运算指令, 例如“增1”、“减1”.
- 或 双目运算: $(A) op(A1) \rightarrow A$ (A 是隐含的CPU某寄存器, 常是累加器)
- ② $(PC)+1 \rightarrow PC$ 。

- 其格式为：

op

操作码op

- 多为“停机”，“空操作”，“清除”等控制类指令。



5.2.3 操作码



- 指令系统中不同的指令 用操作码字段的不同编码OP来表示，每一种编码代表一种指令.
- 1) 定长 操作码
 - 指操作码的长度固定, 且操作码字段在指令中的位置也固定 .
 - 其操作码字段的长度取决于指令系统的规模 n : $\text{Length}_{\text{OP}} = \lceil \log_2 n \rceil$
- 2) 变长 操作码
 - 扩展操作码: 操作码向不用的地址码字段扩展。

扩展操作码 (难)

某指令系统:

双地址指令



最多表示 2^8 条双地址指令

单地址



2^{20} 条

零地址

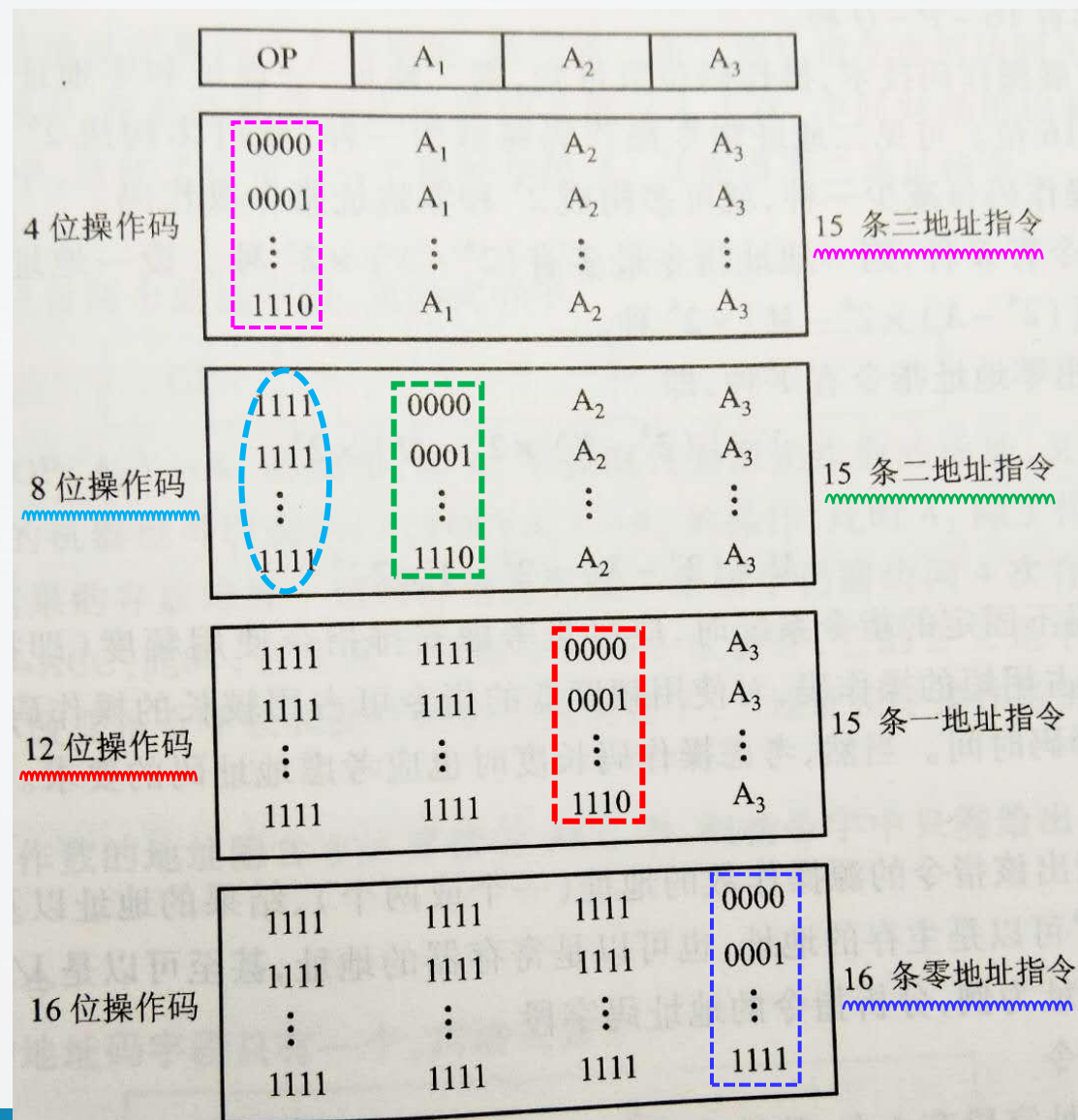


2^{32} 条

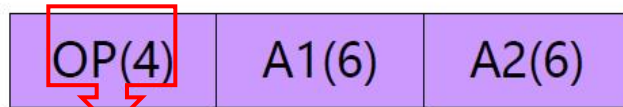
- 这里设: **实际双地址指令的指令条数为 k** , 且 $k < 2^8$
- 此时, $2^8 - k$ 为**多余状态**, 这些多余编码可用作 单地址 或 零地址指令的OP(中的部分)。注意: **双地址指令的OP不能和单地址指令、零地址指令的相同, 否则无法区分。**
- 则, 可用于单地址指令的条数 = $(2^8 - k) \times 2^{12}$, 其中 **2^{12} 是后面紧接的12位可表示的不同编码。**

例1: 若指令字长16位, 操作码字段OP和地址码字段A1、A2、A3各占4位。

- 一种扩展操作码的安排如右图:
- 还有其他多种扩展方法。



例 2 设某指令系统指令字长16位，每个地址码为6位。若要求设计二地址指令15条、一地址指令34条，问最多还可设计多少条零地址指令？



因为：指令字长16位，每个地址码6位(由题：最多2个地址码)

解：操作码按从短码到长码进行扩展编码

假定二地址指令编码：(0000 – 1110) 共15条，1111作为扩展标识；

则一地址指令编码的全集可表示为：1111 000000 – 111111 (假定扩向A1)；

64种编码

因一地址指令只需34条，可将全集中多余的30种编码作为向零地址指令扩展的标识，假定为：100010 – 111111 (指A1字段的6位) OP字段为1111

故最多可支持的 零地址指令数为： 30×2^6 条

A2字段。

综合例：指令格式如下，其中OP为操作码,试分析指令格式的特点。



■ 解：

- 单字长二地址指令
- OP为7位,可以表示128条指令
- 源操作数和目的操作数都是通用寄存器(可分别使用16个)
- 是RR型指令，适合于算术运算和逻辑运算指令

5.3 指令和数据的寻址方式 (重)

- 计算机在运行程序之前 须把指令和数据存放在主存的相应地址单元中。运行程序时，需不断从主存取指令、取数据。
- 由于主存是基于地址访问的存储器，只有获得**指令和操作数在主存中的地址 (称为有效地址)**后，CPU才能访问所需指令、数据。
- 寻址方式 就是寻找指令、操作数的有效地址的方法。

5.3.1 指令的寻址方式

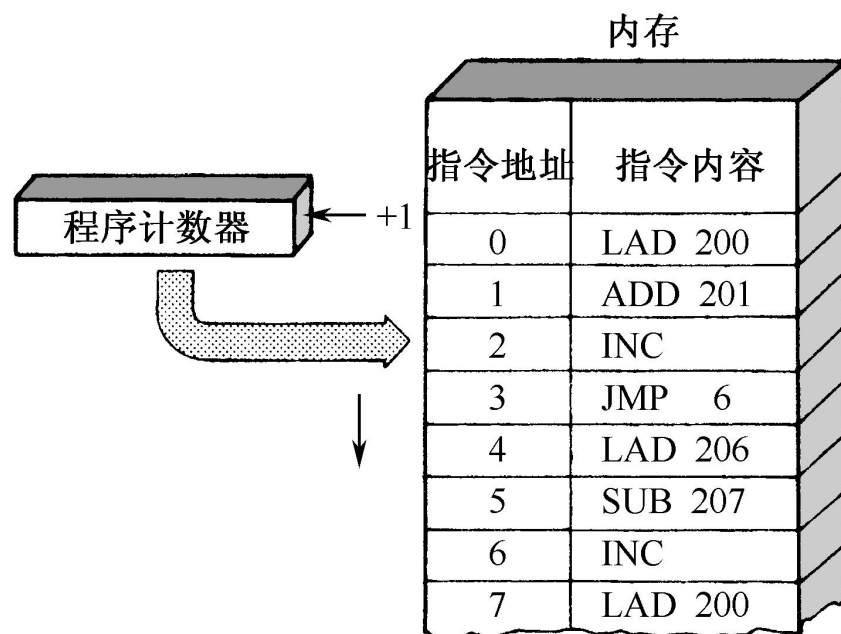
1、顺序寻址

- 一般, 程序的指令序列在主存顺序存放。执行时从第一条指令开始, 逐条取出并逐条执行, 这种程序的顺序执行过程, 称为顺序寻址方式。
- CPU中设置一个程序计数器(PC) 对指令的顺序地址进行计数。开始时, PC中存放程序的首地址, 每执行一条指令, PC 加1, 以指出下条指令的地址, 直到程序结束。
 - PC++ 准确应是加上当前指令的字节长度, 例如 指令字长4B时, 它的值实际是PC+4。
- PC中存放的是下一条指令的地址。

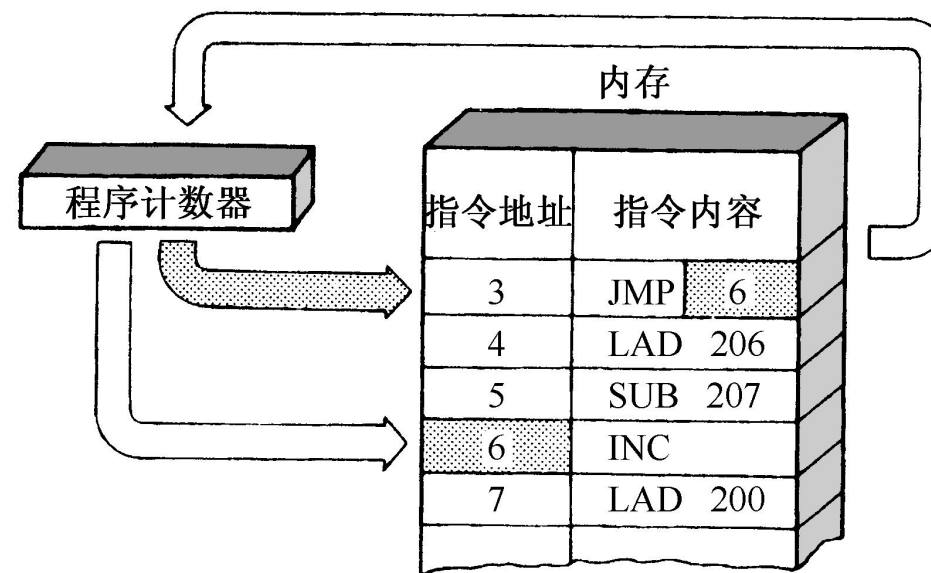
2、跳跃寻址

- 当程序中 出现分支或循环时，就会改变程序的执行顺序，此时采用 跳跃寻址方式。这时下条指令的地址不是直接通过程序计数器PC当前值获得的，而是由(跳转)指令本身给出，并用该值去更新PC。
- 跳跃寻址的处理方式：根据本条指令 重新修改PC的内容，然后进入取下条指令阶段（见下页例）。

- 顺序寻址方式
- 跳跃寻址方式：



(a) 指令的顺序寻址方式



(b) 指令的跳跃寻址方式

- **本子上完成：**
- **1. 试题书P193: 44题(2011考研真题, 其实不难).**
- **2. 教材习题4.17(第(3)问“ 该地址” 指 “ 主存地址”);**
4.18(去掉: 图中TLB表上方的“ 标记 页框号”).
- **3、教材P182习题5.1: 指令, 指令系统, 扩展操作码, 有效地址。**



5.3.2 操作数的寻址方式

- 是形成操作数的有效地址的方法。
 - 以*单地址*指令为例, 地址码的构成如下图。
 - 记: 实际有效地址为**EA**, 实际操作数 **S**, 即 **S = (EA)**。



操作数寻址方式 (重):

- 立即数寻址
- 直接寻址
- 间接寻址
- 寄存器寻址
- 寄存器间接寻址
- 相对寻址
- 变址寻址
- 堆栈寻址。

立即(数)寻址

■ 此时, 地址码字段是操作数本身.

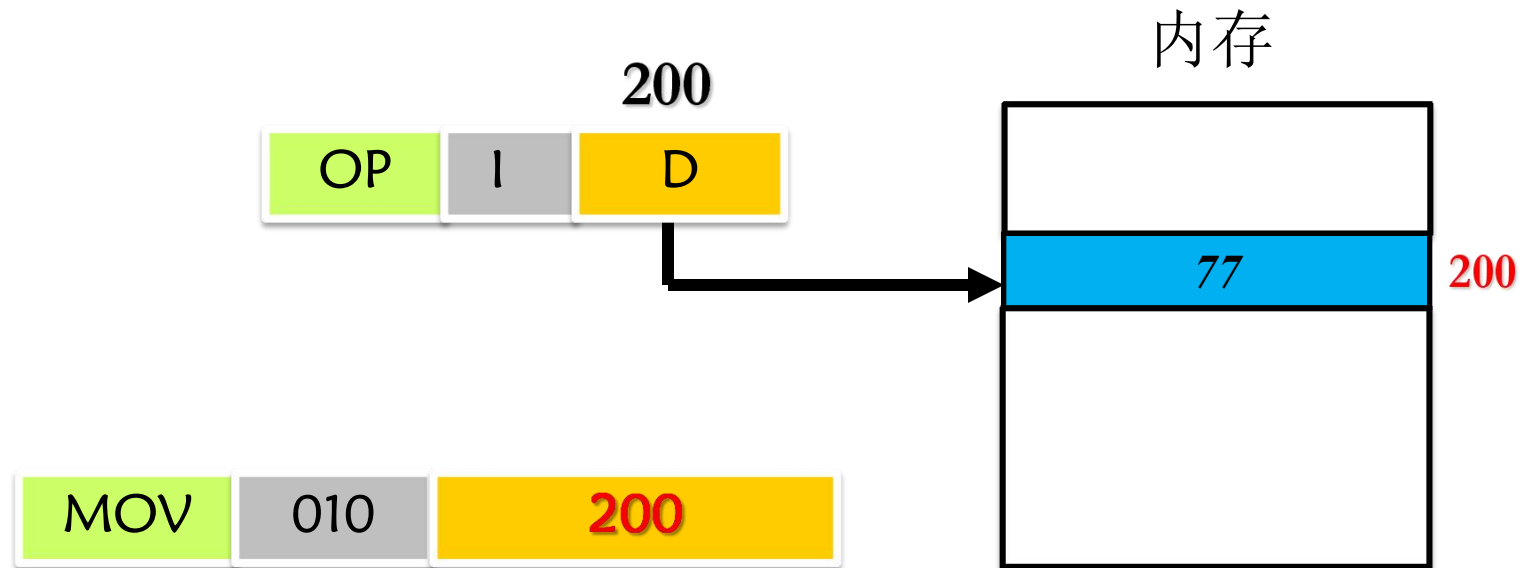
□ S=D

□ 例: **MOV AX, 2038H** (功能: 2038H→AX)



直接寻址（ Direct Addressing ）

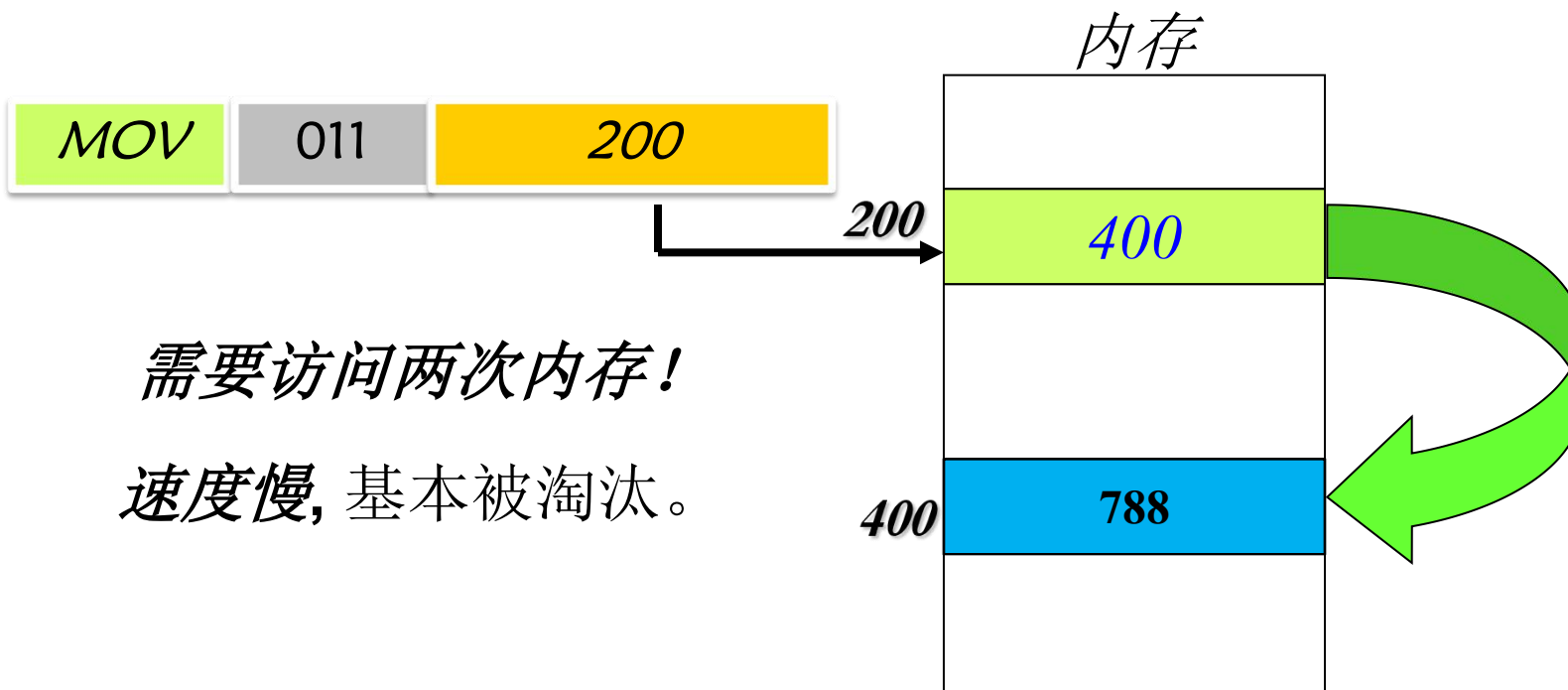
- 地址码字段直接给出操作数在内存的有效地址 **EA: EA=D**
- 例 MOV AX , [200]



间接寻址 (Indirect Addressing)

- **D**字段对应内存空间的内容是操作数地址, 即**D**是操作数地址的地址

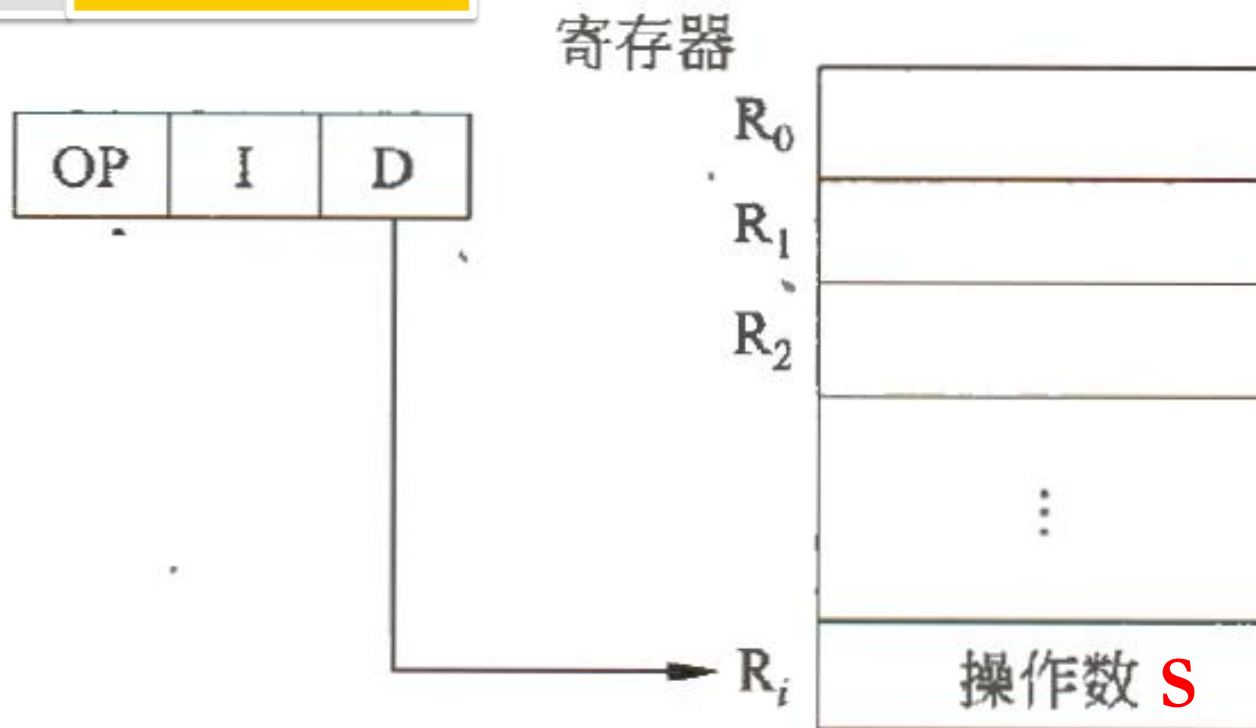
□ **EA=(D)**, **S=(EA)**。下例中 **EA=400**, **S=788**。



寄存器寻址 (Register Addressing)

- 此时操作数 S 在CPU的内部寄存器中。
 - 比如 寄存器AX,BX,CX,DX
 - 例 `MOV AX, BX` (即 $BX \rightarrow AX$) 此时 $EA = Ri$, $S = (Ri)$ 。

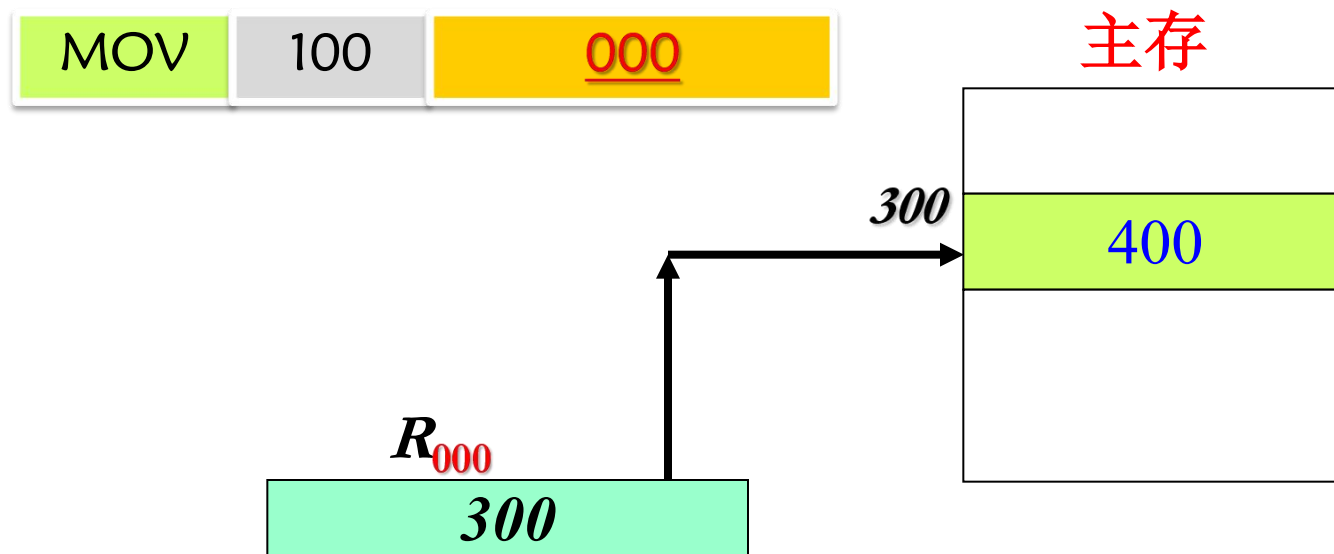
MOV	001	000 (BX寄存器编号)
-----	-----	---------------



寄存器间接寻址 (Register Indirect Addressing)

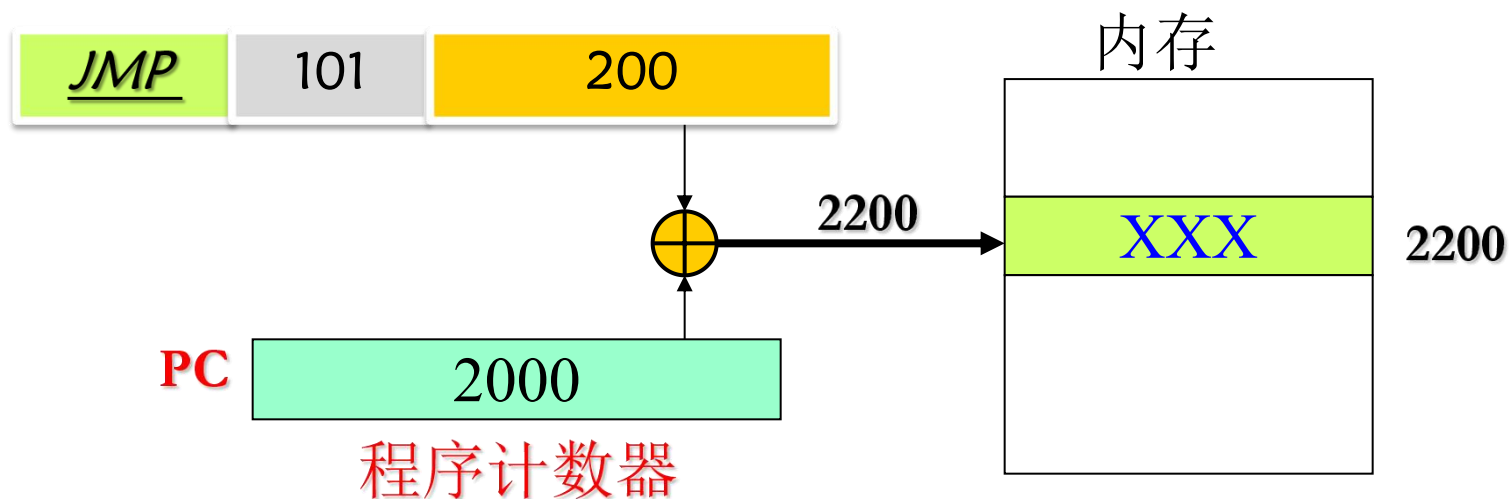
■ D字段内容是 存放操作数主存地址的寄存器 的编号。

□ $EA = (R)$ 例 `MOV AX, [BX]` 其中 $EA=300$, $S=400$ 。



相对寻址 (Relative Addressing)

- 指令中的D 加上 **PC** 的内容，作为操作数的地址 **EA** 。
- $EA = D + PC$, 下例中 $EA = 2200$ 。



P161例5.3

例 某计算机采用双字节长指令,内存基于字节寻址,指令中的数据采用补码表示,且PC的值在取指阶段完成修改。

1)若某采用相对寻址指令的当前地址为2003H,且要求转移后的目标地址为200AH,则该指令形式地址字段的值为多少?

2)若某采用相对寻址的指令的当前地址为2008H,且要求转移后的目标地址为2001H,则该指令的形式地址字段的值为多少?

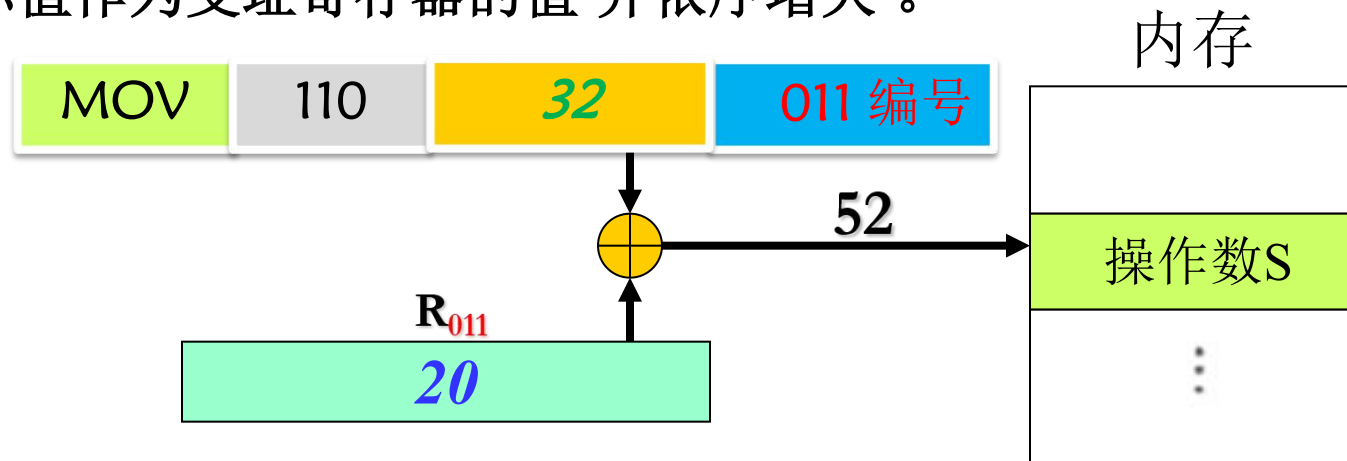
按题目意思(题目上若无此描述,则不加),
PC的当前值应是顺序的下条指令地址

解:1) $D = EA - PC = 200A\text{ H} - (2003\text{ H} + 2) = 5\text{ (0000 0101)}$

2) $2001\text{ H} - (2008\text{ H} + 2) = -9\text{ (1111 0111 即F7H)}$

变址寻址 (Index Addressing)

- 变址寻址是面向用户的, 主要用于解决程序循环问题。
- 一般系统会指定一个变址寄存器R, R的内容可以随要求填入。指令中字段D存放基地址。
- $EA = \underline{D} + (R)$
- 例 MOV AX, 32[SI] SI 为 变址寄存器, 此例 $EA = \underline{32} + \underline{20} = 52$
- 例如数组元素的顺序访问中, 数组首地址 作为D (不变), 元素下标值作为变址寄存器的值 并依序增大。



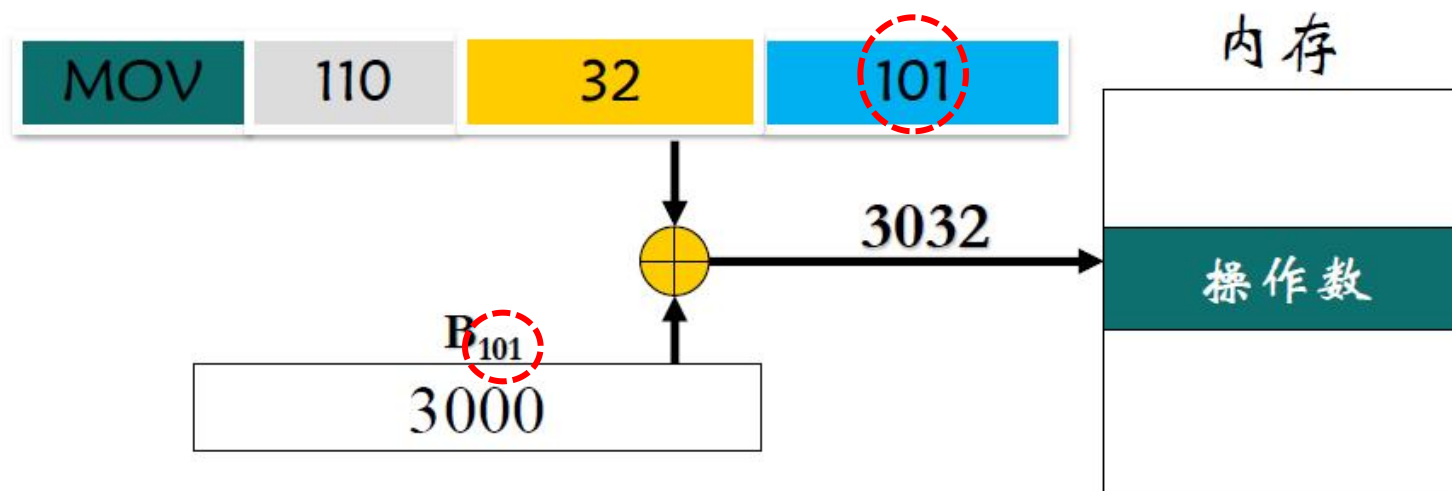
基址寻址

➤ 基址寻址面向系统, 主要用于程序的重定位。例如多道程序设计环境下, 需要由管理程序将多个穿插运行的程序装入主存。

■ 指定一个基址寄存器B, 与本指令地址无关

$E = D + (B)$, D为指令中地址字段的值

■ MOV AX, 32[B]



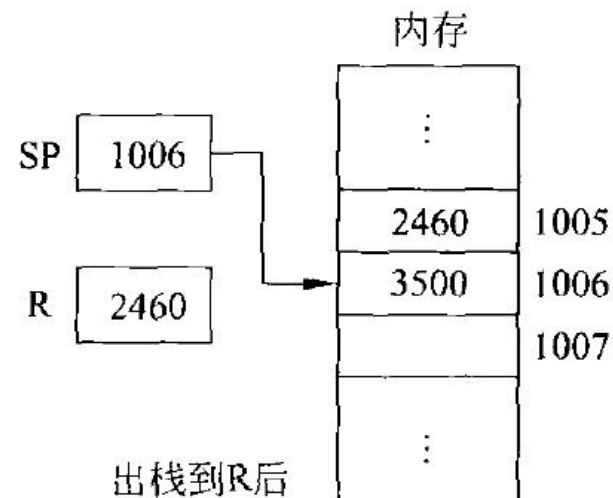
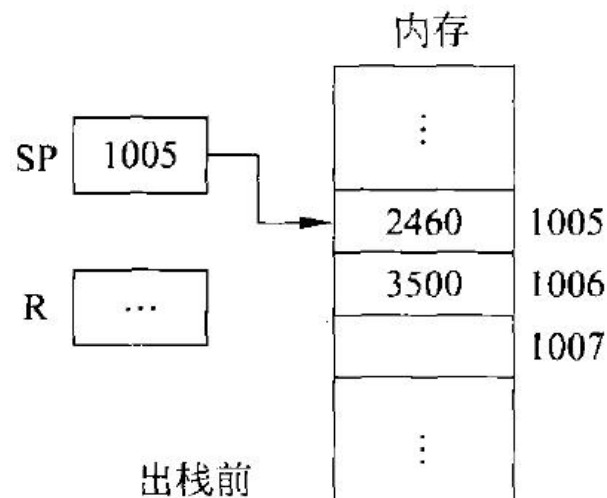
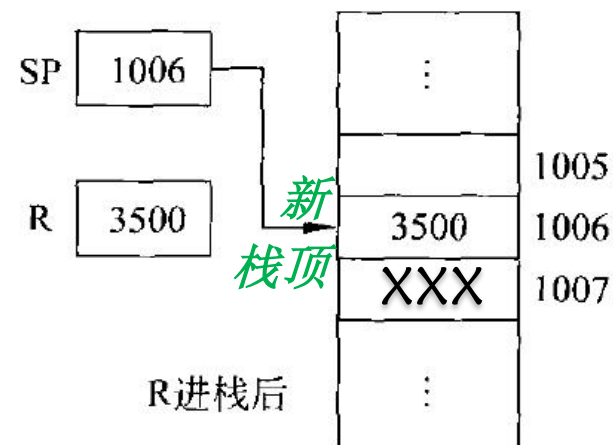
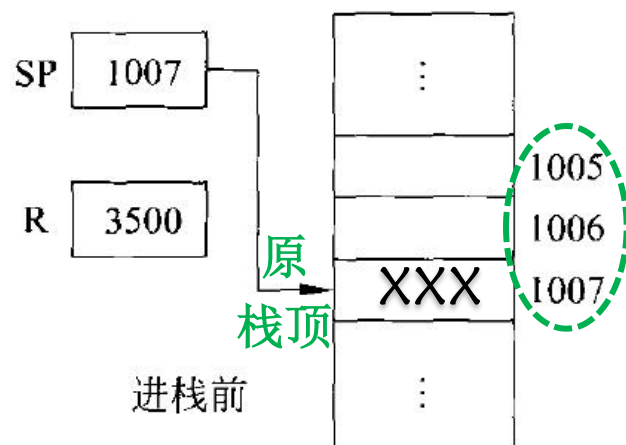
➤ 这里 **D**表示参与基址寻址的偏移量, 后续指令中**D**值会变; 基址寄存器**B**的值在各指令执行过程中不变。与变址寻址正好相反。

堆栈寻址

- 指寻找存放在堆栈中的操作数。
- 堆栈有存储器堆栈 和 寄存器堆栈, 数据的存取都是通过栈顶进行。

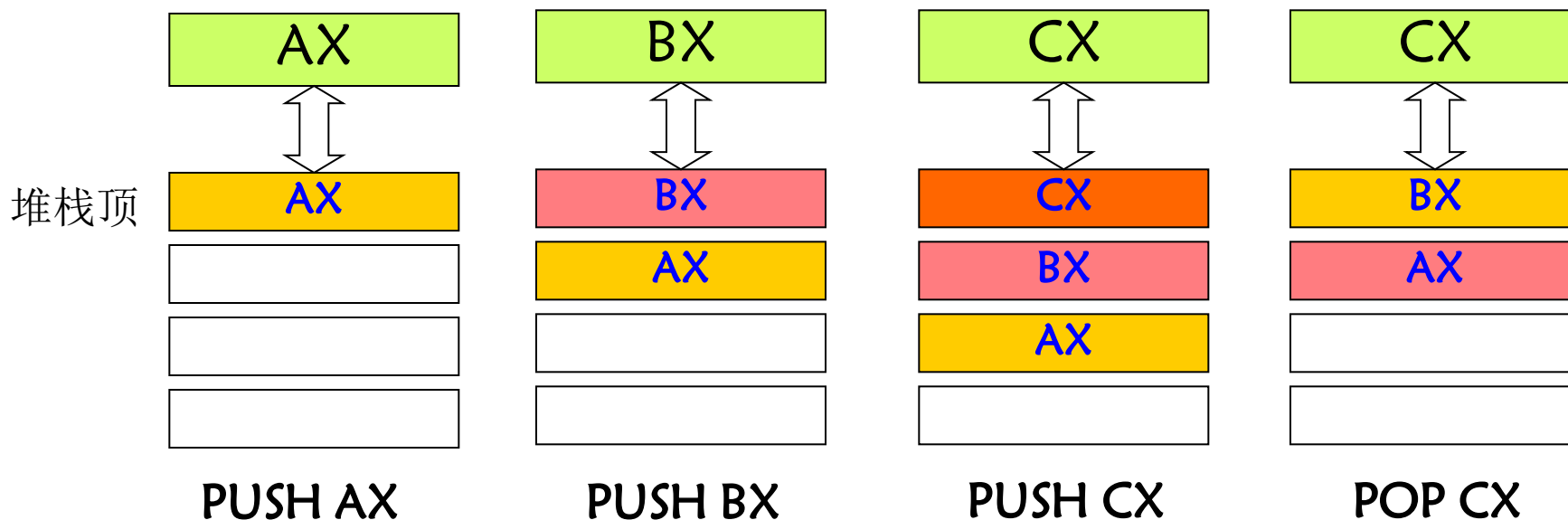
存储器堆栈：

- 由内存区间作堆栈。
- 因内存按地址访问, 所以需: **SP---堆栈指示器 (指示栈顶地址)**。



■ 寄存器堆栈 (P163)

- 是CPU内部一组串联的寄存器
- 数据的传送在栈顶和通用寄存器之间进行
- 无需SP; 进出栈时, 栈顶不动, 数据移动, 进出栈涉及到栈内所有数据的移动
- 栈容量有限, 取决于堆栈寄存器的数量。



栈顶不动, 数据移动。

常见的数据寻址方式小结

立即寻址	$S=D$	快, 便于程序设计, 赋初值, 初值大小受限
寄存器寻址	$S=(R)$	快, 便于程序设计, 不能访问主存
直接寻址	$E=D$	慢, 便于程序设计, 提供访存, 范围受限
间接寻址	$E=(D)$	很慢, 解决直接寻址访存范围受限的问题
寄存器间接	$E=(R)$	慢, 便于程序设计, 提供访存, 范围增大
相对寻址	$E=(PC)+D$	慢, 提供访存, 不能在循环中使用
变址寻址	$E=(X)+D$	慢, 便于程序设计, 提供访存, 可在循环中用
基址寻址	$E=(B)+D$	慢, 提供更大的范围的访存能力, 不能在循环中用

使用寻址方式的好处： 有利于缩短指令字长、方便程序设计、扩展访存空间！



例: 设某机的指令格式、有关寄存器和主存内容如下, **X**表示寻址方式, **D**为形式地址, 请在下表中填入有效地址**EA**及操作数的值。

OP	X	D=100
----	---	-------

下条指令地址PC=1000

$R_{\text{变}} = 2000$

100	200
200	500
500	800
1100	100
2100	200

寻址方式	X	有效地址EA	操作数
立即数	0	$S=D$	100
直接	1	$EA=D=100$	200
间接	2	$EA=(D)=200$	500
相对	3	$EA=(PC)+D=1100$	100
变址	4	$EA=(R)+D=2100$	200
变址间址(P164)	5	$EA=((R)+D)=200$	500

[例] 一种二地址RS型指令的结构如下：

6位 4位

OP 通用寄存器

1位 2位 16位

I X 偏移量D

其中 I 为间接寻址标志位，X为寻址模式字段，D为偏移量字段。通过 I，X，D的组合，可构成如下寻址方式：

寻址方式	I	X	有效地址E算法	说明
(1)	0	00	$E=D$	
(2)	0	01	$E=(PC) \pm D$	PC为程序计数器
(3)	0	10	$E=(R_2) \pm D$	R_2 为变址寄存器
(4)	1	11	$E=(R_3)$	
(5)	1	00	$E=(D)$	
(6)	0	11	$E=(R_1) \pm D$	R_1 为基址寄存器

请写出6种寻址方式的名称。

- 答：
 - (1)直接寻址
 - (2)相对寻址
 - (3)变址寻址
 - (4)寄存器间接寻址
 - (5)间接寻址
 - (6)基址寻址。

□ 数据传送类指令

- ◆ 取数 `MOV AX, TEMP` (TEMP主存中某地址(的数据) → AX)
- ◆ 存数 `MOV TEMP, AX`
- ◆ 传送 `MOV AX, CX`

□ 运算类指令：算术运算指令、逻辑运算指令、移位指令

□ 程序控制类指令：用于控制程序的执行方向，并使程序具有测试、分析与判断的能力。例如，转移指令。

□ 输入/输出类：

- ◆ `IN AX, n` (n: 外设某输入端口的地址) 。

5.4 指令类型

- **字符串处理**：字符串转换、比较、查找等指令
- **特权指令**：系统资源的分配和管理，只用于OS或其他系统软件中。
- **其他指令**：测试指令、暂停指令等。

5.5 指令格式设计 (重)

■ 指令格式设计的主要内容:



- 根据指令数量的要求及是否支持操作码扩展，确定操作码字段的位数
- 根据对操作数的要求确定地址码字段的个数
- 根据寻址方式的要求，为每个地址码字段确定寻址方式字段位数
- 确定采用定长指令还是变长指令

P167例5.4

例1.某机字长16位,主存64KB,指令单字长 单地址, 80条指令。寻址方式有 直接、间接、相对、变址。请设计指令格式。

解：80条指令 $\lceil \log_2 80 \rceil = 7$ 采用7位操作码

4种寻址方式 $\Rightarrow$ 寻址方式特征位2位

单地址位长度: $16 - 7 - 2 = 7$ 位



例5.4



假设 存储字长(存储单元的长度) 也是16位

设PC寄存器16位 变址寄存器16位

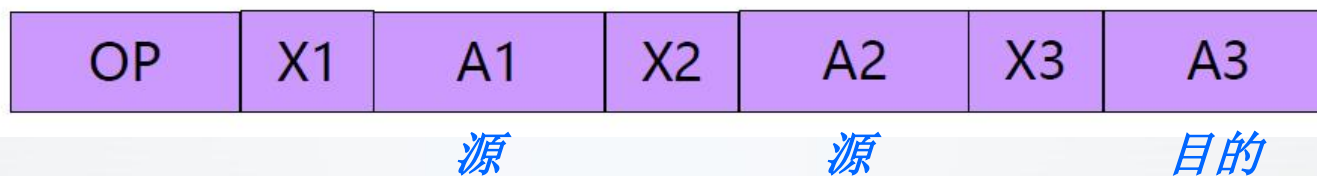
- 相对寻址 $E = (PC) + A$, 寻址范围为 : 64K (2^{16})
- 变址寻址 $E = (R) + A$, 寻址范围为 : 64K
- 直接寻址 $E = A$, 寻址范围为 : 128 (2^7)
- 间接寻址 $E = \underline{(A)}$, 寻址范围为 : 64K (2^{16})

例 某机字长32位，采用三地址指令，支持8种寻址操作，完成60种操作，各寻址方式均可在2K主存范围内取得操作数，并可在1K范围内保存运算结果。问应采用什么样的指令格式？指令字长最少应为多少位？

源操作数

目的操作数

解：根据题目条件，指令格式如下：



(字段位数) OP = 6位

寻址方式位数 X1 = X2 = X3 = 3, 共 9位

按题意,需考虑 A1=A2=11, A3=10位, 共 32位

⇒ 47位

5.6 CISC和RISC基本概念

请自学这3页

□ CISC和RISC是CPU的两种架构，其指令系统的复杂程度不同。

■ CISC---复杂指令系统计算机

□ Complex Instruction System Computer

□ 指令数量多，指令功能复杂的计算机。

□ 例: Intel X86

■ RISC---精简指令系统计算机

□ Reduced Instruction System Computer

□ 指令数量少，指令功能单一的计算机。

□ 例: *MIPS*指令系统。

- **CISC指令系统主要特点:** 1)指令系统庞杂, 指令数目常多达二三百条. 2)寻址方式多. 3)指令格式多, 指令字长不固定. 4)访存指令不加限制. 5)大多数采用 微程序控制器(见《处理器》).
- CISC中, 通过增强指令系统的功能, 简化了软件, 但增加了硬件的复杂程度。然而指令复杂了, 指令的执行时间必然加长, 从而使整个系统的执行时间反而增加.
- 20%和80%规律: CISC中只有约20%的指令使用频率高, 占据了80%的处理机时间。

精减指令系统 (RISC)

- 选取使用频率最高的一些简单指令,指令条数少;
- 寻址方式简单
- 指令长度固定,指令格式简单
- CPU设置大量寄存器; 只有存/取数指令才访问存储器, 其余指令的操作都在寄存器之间进行.
- 每一个机器周期完成一条机器指令。

MIPS指令系统 (重)



MIPS指令系统概述

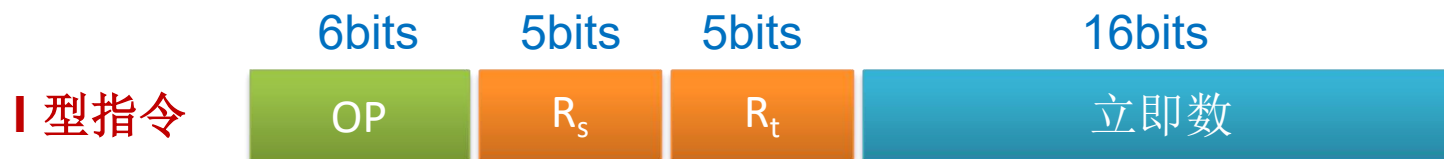
- MIPS (Microprocessor without Intellocked Pipeline Stages)是80年代初期由斯坦福大学Hennessy教授领导的研究小组研制成功; **Million Instructions Per Second**
- 属于精简指令集计算机RISC(Reduced Instruction Set Computer);
- MIPS指令集有以下特点:
 - ◆ 简单的Load/Store结构
 - ◆ 易于流水线CPU设计
 - ◆ 易于编译器开发
 - ◆ MIPS指令的寻址方式非常简单, 每条指令的操作也非常简单
- MIPS包括32个32位的通用寄存器; 这里 指令字长: 32bits= 1Word
 - \$0, \$1, \$2, ... \$30, \$31 。

MIPS指令格式

只有3种指令格式:



◆ R_s, R_t 分别为第一、二源操作数; R_d 为目标操作数;



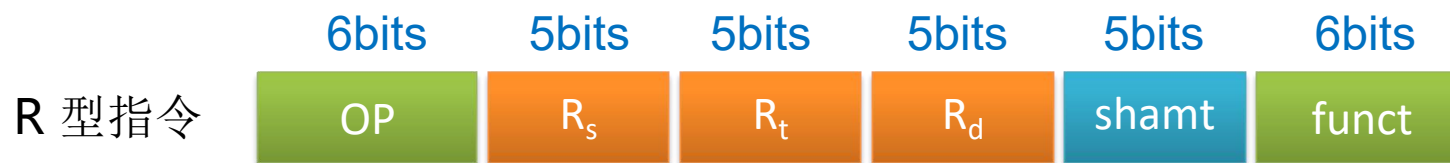
- ◆ 双目、Load/Store: R_s 和立即数是源操作数, R_t 为目标操作数;
- ◆ 条件转移: R_s, R_t 均为源操作数;



◆ 26位立即数作为跳转目标地址的部分地址



32位定长 MIPS指令格式（R型指令）



- OP: 指令的基本操作---操作码
- R_s : 第一个源操作数所在的寄存器编号
- R_t : 第二个源操作数寄存器编号
- R_d : 存放结果的目的地操作数寄存器
- Shamt: 偏移量, 用于移位指令
- Funct: 功能码, 对操作码进行补充。

MIPS指令格式 (部分 *R型* 指令)

指令	格式	6bits OP	5bits rs	5bits rt	5bits rd	5bits shamt	6bits funct
add	R	<i>0</i>	Reg	Reg	Reg	0	<i>32</i> ₁₀
sub	R	<i>0</i>	Reg	Reg	Reg	0	34 ₁₀
and	R	<i>0</i>	Reg	Reg	Reg	0	36 ₁₀
or	R	<i>0</i>	Reg	Reg	Reg	0	37 ₁₀
<u>sll</u>	R	<i>0</i>		<i>Reg</i>	<i>Reg</i>	<i>x</i>	0 ₁₀
srl	R	<i>0</i>	Reg	Reg		x	2 ₁₀
jr	R	<i>0</i>	<i>Reg</i>			0	8 ₁₀

add	R	0	18	19	17	0	32
-----	---	---	----	----	----	---	----

■ 例: **add \$s1,\$s2,\$s3**

指令功能: $\$rd \leftarrow \$rs \text{ op } \$rt$

32位定长MIPS指令格式（I型指令）



操作数中涉及立即数，结果保存到寄存器；

- ◆ op: 标识指令的操作功能；
- ◆ rs: 第1个源操作数，是寄存器操作数；
- ◆ rt: 目的寄存器编号，用来保存运算结果；
- ◆ imm: 第2个源操作数，立即数；

MIPS指令格式 (部分 I、J型指令)

		6bits	5bits	5bits	16bits
指令	格式	OP	rs	rt	imm
<u>addi</u>	I	8	Reg	Reg	16bits 立即数
lw	I	35	Reg	Reg	16bits 立即数
sw	I	43	Reg	Reg	16bits 立即数
andi	I	12	Reg	Reg	16bits 立即数
beq	I	4	Reg	Reg	16bits 立即数 (相对寻址)
bne	I	5	Reg	Reg	16bits 立即数 (相对寻址)

<i>j</i> 无条件跳转	J	2	26bit 立即数 (伪直接寻址)		
jal	J	3	26bit 立即数 (伪直接寻址)		

I 型指令

◆ 面向运算的I型指令

指令	[31:26]	[25:21]	[20:16]	[15:0]	指令功能
addi	001000	rs	rt	imm	寄存器和立即数 “加”
andi	001100	rs	rt	imm	寄存器和立即数 “与”
ori	001101	rs	rt	imm	寄存器和立即数 “或”
xori	001110	rs	rt	imm	寄存器和立即数 “异或”

■ 格式及功能:

需 (E)扩展成32位

addi/andi/ori/xori rt, rs, imm; # \$rt ← \$rs op E(imm)

第一条指令是进行符号扩展，其余是0扩展 (0扩展不同于符号扩展，即无符号扩展)。

I 型指令

◆ 面向访存的I型指令

指令	[31:26]	[25:21]	[20:16]	[15:0]	指令功能
lw	100011	rs	rt	imm	从存储器中读取数据
sw	101011	rs	rt	imm	把数据保存到存储器

MIPS32中唯一两条访问存储器的指令

基址寄存器 偏移量

lw rt, imm(rs) # \$rt $\leftarrow$ mem[\$rs + E(imm)]

sw rt, imm(rs) # mem[\$rs + E(imm)] $\leftarrow$ \$rt

- 对imm要扩展成**32**位, 是**带符号**的扩展。

I 型指令

◆ 面向条件转移(分支)的I型指令

指令	[31:26]	[25:21]	[20:16]	[15:0]	指令功能
beq	000100	rs	rt	imm	寄存器相等则转移
bne	000101	rs	rt	imm	寄存器不等则转移

beq rs, rt, imm #if(\$rs==\$rt) $PC \leftarrow PC + E(imm) \ll 2$

bne rs, rt, imm #if(\$rs!=\$rt) $PC \leftarrow PC + E(imm) \ll 2$

是标准的PC相对寻址方式

其中imm要先 “带符号扩展” 成32位, 再左移2位。

MIPS寻址方式

- 在MIPS32指令集中，不单设寻址方式说明字段

◆ R型指令：由op和funct字段共同隐含说明当前的寻址方式；



◆ I型和J型指令：由op字段隐含说明当前指令使用的寻址方式。

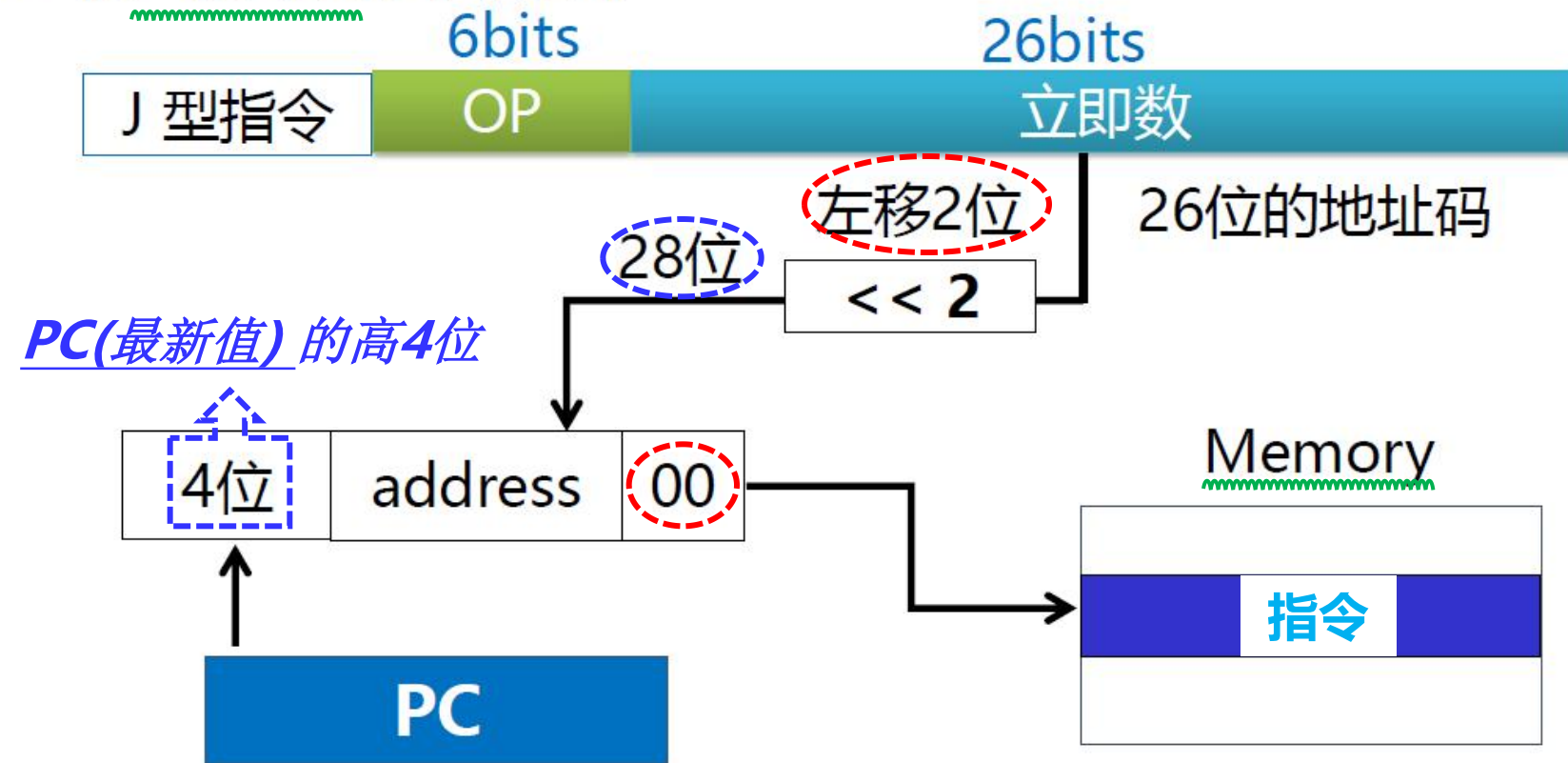


■ 立即数寻址 (Immediate addressing)



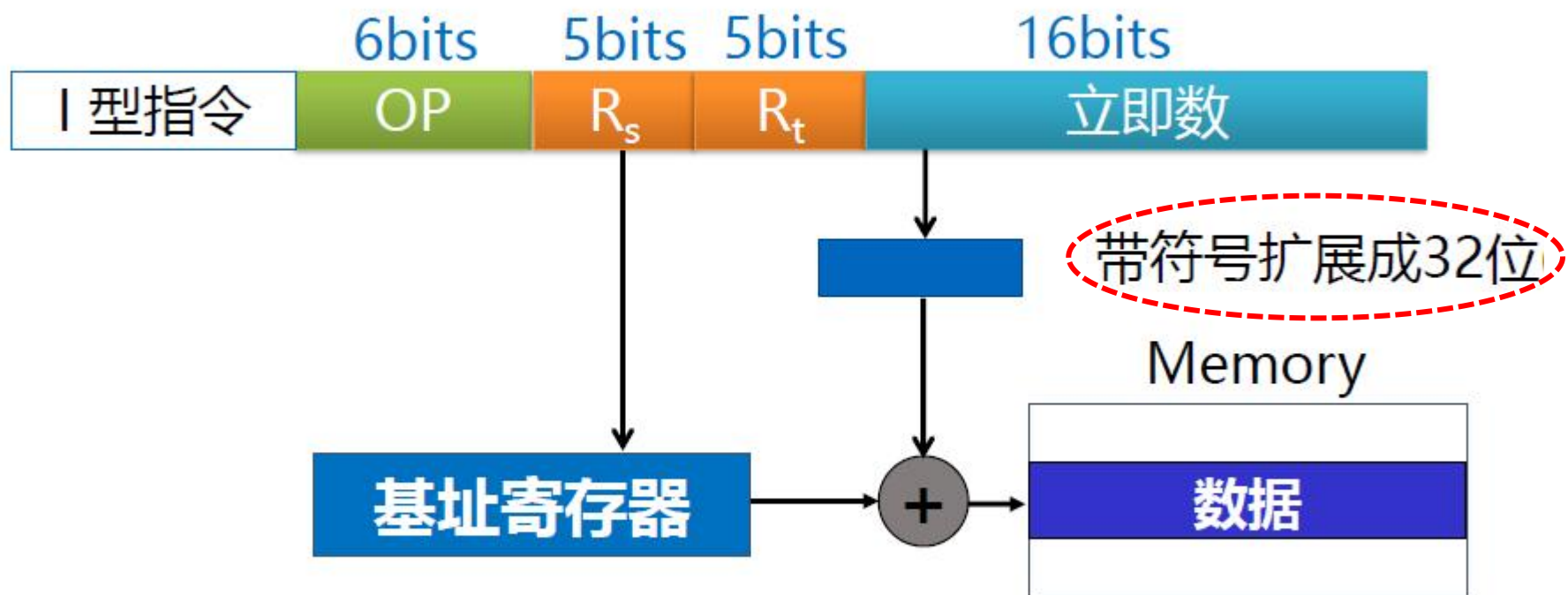
addi s1, s2, 10 ($\$s1 \leftarrow \$s2 + E(10)$)

■ 伪直接寻址(页面寻址)



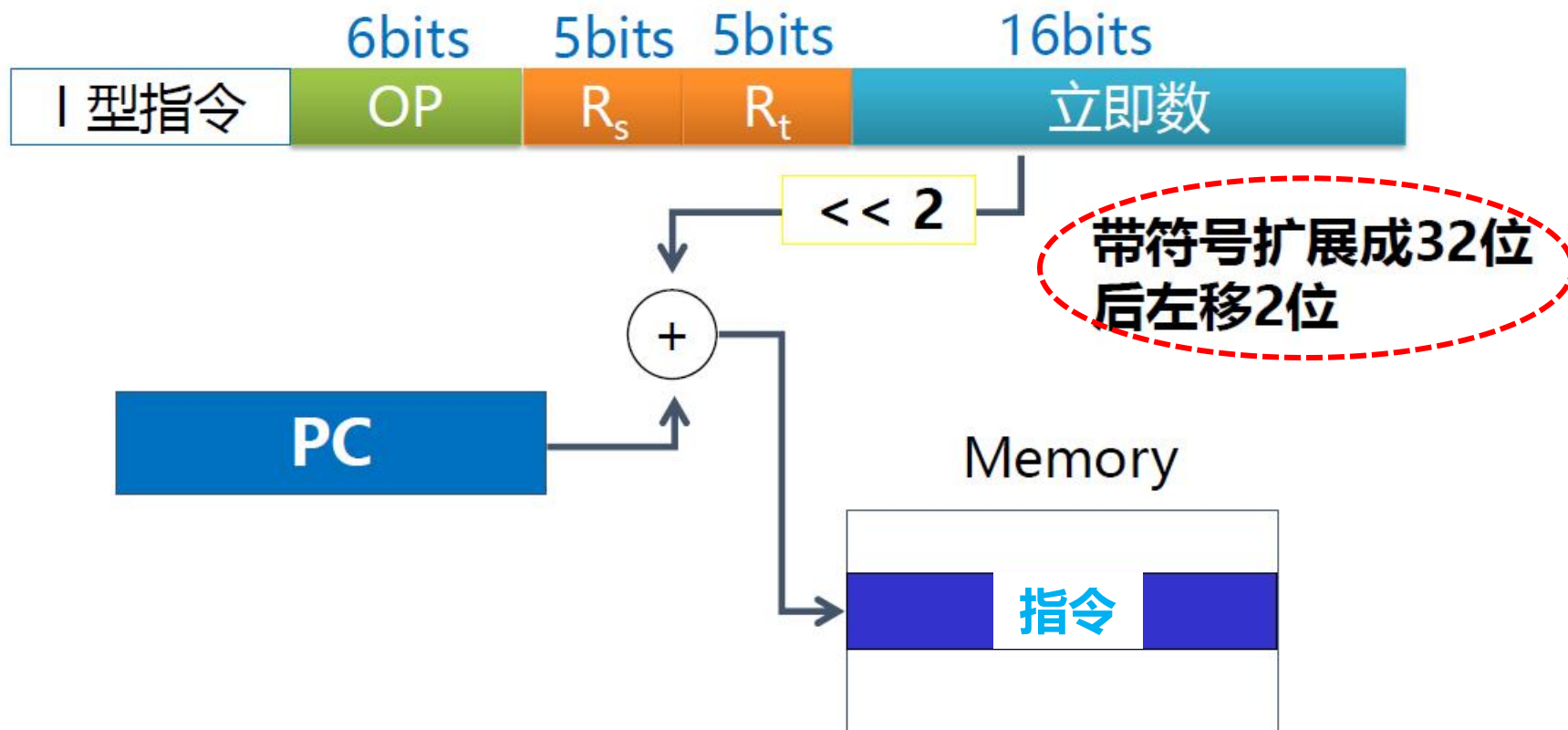
■ 使用伪直接寻址的指令: j, jal

■ 基址寻址(Basic Addressing)



■ 使用基址寻址的指令: lw ,sw, lh, sh, lb, lbu等

■ 相对寻址



■ 使用相对寻址的指令: beq, bne

■ 寄存器直接寻址(Register Addressing)



add \$t0,\$s1,\$s2 ($\$t0 = \$s1 + \$s2$)

MIPS寻址方式小结

- 寄存器寻址
- 立即数寻址
- 基址寻址
- 相对寻址
- 伪直接寻址：例如 `J label` 如下

PC 高4位

26位立即数

00

- 具体查阅网课：中国大学MOOC《计算机组成原理》(华中科技大学)。

- ✓ 我国龙芯中科公司研发的龙芯**CPU**, 其早期使用的是**MIPS**指令系统 (并未完全买断), 但她现已推出并使用了完全自主可控的**LoongArch**指令系统 .
- ✓ 同学们也应有投身 *国产IT生态自主可控事业* 的自信和决心 !

作业 2

- 在试题书上完成4.1选择题: 1--6, 8,9,10,15--18.
- 本子上完成:
- 1、教材P182习题5.1: CISC , RISC。 P183习题5.3 (4)(6)(7).
- 2、试题书4.3设计题: 2, 9(试题书上答案的结果是对的,但过程写得并不好, 因此大家自己作答)
- 3、教材P184习题5.8 (“设机器按字节编址” , 题目改为 “ $I=00$: 直接寻址”)
- 4、试题书4.2分析题: 2 (提示: “ 位移量” 即形式地址D), 4(M: 0改为 “ 直接寻址” ; 不做第(3)问)。

The End

本章重点内容：

- 指令系统基本概念
- 指令基本格式
- 指令系统寻址方式。